

Dispensa_Completa_Kernel_Hacking

Dispensa Completa di Kernel Hacking

Corso di Laurea Magistrale in Informatica – Università di Modena e Reggio Emilia
Questo documento unisce e approfondisce tutti i moduli del corso, creando un filo logico tra gli argomenti per facilitare lo studio e la memorizzazione.

Il Filo Logico del Corso

Il corso segue un percorso naturale che rispecchia il ciclo di vita di un kernel hacker:

flowchart LR

```
A["1. COSTRUZIONE<br/>(Mod 01)"] → B["2. ANATOMIA<br/>(Mod 02)"]  
B → C["3. ESTENSIONE<br/>(Mod 03)"]  
C → D["4. DEBUGGING<br/>(Mod 04)"]  
D → E["5. MONITORAGGIO<br/>(Mod 05)"]  
E → F["6. GESTIONE<br/>(Mod 06)"]
```

1. **Costruzione (Modulo 01):** Prima di tutto, impariamo a *costruire* lo strumento di lavoro – il kernel – partendo dai sorgenti.
2. **Anatomia (Modulo 02):** Una volta compilato, capiamo *com'è fatto* l'oggetto prodotto (vmlinux, initramfs) e *come si avvia*.
3. **Estensione (Modulo 03):** Ora che funziona, impariamo a *modificarlo* aggiungendo codice nostro (moduli) e capiamo come il kernel scandisce il tempo (HZ/jiffies).
4. **Debugging (Modulo 04):** Modificare il kernel è pericoloso. Se si rompe, dobbiamo capire *perché* (OOPS). Se funziona, dobbiamo poterci *comunicare* (Sysfs).
5. **Monitoraggio (Modulo 05):** Oltre al Sysfs, esiste un modo più diretto per chiedere servizi (Syscall) e per *controllare le prestazioni* (Tracing).
6. **Gestione (Modulo 06):** Tutto questo lavoro va *gestito professionalmente* e condiviso con la community (Git, patch).

PARTE I: L'Ambiente e le Basi

Modulo 01: Introduzione e Compilazione del Kernel

1.1 Filosofia del Kernel Hacking

Il Kernel Linux è un progetto in continua evoluzione. I libri di testo diventano obsoleti in pochi mesi. Le uniche fonti di verità affidabili sono:

Fonte	Descrizione	Affidabilità
Il Codice	"The code is the documentation" – Spesso è l'unica fonte di verità	★★★★★
Mailing List	Discussioni tra sviluppatori (LKML)	★★★★★
kernel.org	Sito ufficiale del progetto	★★★★★
docs.kernel.org	Documentazione ufficiale (ma non sempre aggiornata)	★★★★
Forum/Blog	Utili ma da verificare – possono essere obsoleti	★★★

Approccio Mentale Richiesto:

- La **Legge di Murphy** è particolarmente attiva nel kernel hacking
- Gli imprevisti sono la norma, non l'eccezione
- *Sapersela cavare* è la skill principale da sviluppare
- Incrociare sempre più fonti prima di fidarsi

1.2 Esplorare il Sistema: La Directory `/boot`

Prima di compilare un kernel, è importante capire dove "vive" quello attualmente in esecuzione:

```
ls -la /boot/
```

File chiave per ogni versione di kernel installata:

File	Descrizione	Utilizzo
<code>vmlinuz-<versione></code>	Immagine compressa del kernel	Caricato in RAM all'avvio
<code>initrd-<versione></code> o <code>initramfs-<versione></code>	Initial RAM Disk	Filesystem temporaneo per l'avvio
<code>config-<versione></code>	Configurazione di compilazione	Riferimento per ricompilare
<code>System.map-<versione></code>	Tabella dei simboli	Debug degli indirizzi di memoria

Comando fondamentale:

```
uname -a # Mostra versione e data di compilazione del kernel in esecuzione
```

1.3 Procedura Completa di Compilazione

1.3.1 Prerequisiti

Ambiente consigliato: Macchina Virtuale (VirtualBox o KVM/QEMU)

- Disco: almeno 30GB liberi
- RAM: almeno 4GB
- Processori: quanti più possibile (accelerano la compilazione)

Installazione dipendenze:

Debian/Ubuntu:

```
sudo apt install git build-essential libncurses5-dev wget bzip2 \
libssl-dev libelf-dev flex bison dwarves
```

Fedora/RHEL:

```
sudo dnf groupinstall "Development Tools"
sudo dnf install ncurses-devel wget bzip2 openssl openssl-devel \
elfutils-libelf-devel bc dwarves gcc
```

1.3.2 Ottenere i Sorgenti

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
```

1.3.3 Configurazione

La configurazione determina quali componenti includere nel kernel. Esistono due approcci principali:

Approccio 1: Configurazione Minima (Consigliato per iniziare)

```
make defconfig          # Crea config di default
./scripts/kconfig/streamline_config.pl > minimal_config
mv minimal_config .config
```

Lo script `streamline_config.pl` analizza i moduli *attualmente in uso* e crea una configurazione ottimizzata. **Importante:** eseguirlo con tutti i dispositivi che si useranno collegati e i moduli necessari già caricati.

Approccio 2: Copiare la Configurazione del Kernel Running

```
# Se disponibile:
zcat /proc/config.gz > .config
# Oppure:
cp /boot/config-$(uname -r) .config
```

Approccio 3: Configurazione Interattiva

```
make menuconfig  # Interfaccia ncurses per modificare le opzioni
```

1.3.4 Compilazione

```
make -j $((nproc) + 1)
```

⚠️ **Attenzione:** Non usare MAI sudo per la compilazione! Rischio di corrompere i permessi.

Se la compilazione fallisce:

Errore	Soluzione
<code>openssl/opensslv.h: No such file</code>	<code>apt install libssl-dev</code> (o <code>dnf install openssl-devel</code>)
Errori strani/corruzione	<code>make mrproper</code> (pulisce tutto), poi riconfigurare
Spazio insufficiente	Liberare spazio o usare cartella di build separata

1.3.5 Installazione

```
sudo make modules_install # Installa moduli in /lib/modules/<versione>
sudo make install          # Installa kernel in /boot e configura GRUB
```

Per distribuzioni Fedora-like (se GRUB non si aggiorna):

```
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
# oppure per EFI:
sudo grub2-mkconfig -o /boot/efi/EFI/fedora/grub.cfg
```

1.3.6 Post-Installazione

1. Riavviare il sistema
2. Al menu GRUB, selezionare il nuovo kernel
3. Verificare: `uname -a` (controllare versione e data)

Se il kernel non si avvia:

- Rimuovere `quiet` e `splash` dalla riga di comando GRUB per vedere i messaggi di errore
- Problemi comuni: driver disco mancante, filesystem non supportato

1.4 Gestione dei Kernel Installati

Rimuovere un kernel custom:

```
sudo rm /boot/*<versione_kernel>*
sudo rm -rf /lib/modules/*<versione_kernel>*
sudo update-grub
```

Modulo 02: Concetti Base e Processo di Avvio

2.1 Struttura del Kernel nel Filesystem

Il kernel in esecuzione è presente in due “forme” nel filesystem:

1. Forma Statica (in `/boot`):

- `vmlinuz`: L'immagine compressa (formato bzImage)

- **initramfs** : Archivio cpio compresso con driver essenziali
- **config** : La configurazione usata per compilarlo

2. Forma Dinamica (in **/lib/modules**):

- Moduli caricabili (.ko files)
- Metadata sui moduli (dependencies, aliases)

2.2 Il Paradosso dell'Avvio e l'Initramfs

Il Problema: Il kernel ha bisogno dei driver per accedere al disco, ma i driver sono *sul disco*. Come risolverlo?

La Soluzione: Initramfs (Initial RAM Filesystem)

flowchart TD

```

A["1. BIOS/UEFI"] → B["2. Bootloader - GRUB"]
B → C["3. Kernel + Initramfs"]
C → D["4. Kernel scompatta initramfs in RAM"]
D → E["5. Esegue /init dall'initramfs"]
E → F["6. /init carica driver - storage, fs"]
F → G["7. /init monta vero root filesystem"]
G → H["8. Pivot root → systemd"]

```

Contenuto tipico dell'initramfs:

- Driver per storage (SATA, NVMe, SCSI)
- Driver per filesystem (ext4, xfs, btrfs)
- Driver HID (tastiera, mouse)
- Script di inizializzazione (**/init**)

Comandi utili:

```

lsinitramfs /boot/initrd.img-$(uname -r)      # Lista contenuto
unmkinitramfs /boot/initrd.img-$(uname -r) /tmp/ # Scompatta
update-initramfs -u                            # Rigenera

```

2.3 Moduli del Kernel: Built-in vs Loadable

I componenti del kernel possono essere compilati in due modi:

Tipo	Simbolo Config	Descrizione	Pro	Contro
Built-in	y (=yes)	Integrato in vmlinuz	Sempre disponibile	Occupava sempre RAM
Modulo	m (=module)	File .ko separato	Caricato on-demand	Richiede initramfs per boot
Disabilitato	n (=no)	Non compilato	Risparmio spazio	Non disponibile

File importanti:

```
/lib/modules/$(uname -r)/modules.builtin # Lista moduli built-in
/lib/modules/$(uname -r)/kernel/        # Albero dei moduli loadable
```

Comandi per gestire i moduli:

```
lsmod          # Lista moduli caricati
modprobe <nome> # Carica un modulo
modprobe -r <nome> # Rimuove un modulo
modinfo <nome>  # Info su un modulo
```

Aspetto critico: Sia i moduli built-in che loadable condividono lo *stesso spazio di indirizzamento* del kernel. Un bug in un modulo può corrompere l'intero sistema!

2.4 Il Processo di Avvio Dettagliato

flowchart TD

```
A["Power On"] → B["1. BIOS/UEFI<br>POST, init hardware"]
B → C["2. Bootloader - GRUB<br>Carica vmlinuz + initramfs"]
C → D["3. start_kernel<br>Decompressione, init CPU/memoria"]
D → E["4. rest_init<br>Crea kernel threads"]
E → F["5. /init - initramfs<br>Carica driver, monta root"]
F → G["6. /sbin/init - PID 1<br>Avvia systemd/servizi"]
```

Per esplorare il codice:

- Usare **Elixir Cross Referencer**: <https://elixir.bootlin.com/linux/latest/source>
- Funzioni chiave: `start_kernel` → `rest_init` → `kernel_init` → `run_init_process`

PARTE II: Estendere il Kernel

Modulo 03: Kbuild e Creazione di Moduli

3.1 Anatomia di un Modulo Kernel

Un modulo kernel è un pezzo di codice che può essere caricato e scaricato a runtime. Ha una struttura precisa:

```
// hello_module.c
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt // Prefisso per i log

#include <linux/init.h> // Per __init e __exit
#include <linux/kernel.h> // Per pr_info, pr_err, etc.
#include <linux/module.h> // Per module_init, module_exit, MODULE_*

// Funzione di inizializzazione - chiamata al caricamento
```

```
// __init: la memoria viene liberata dopo l'esecuzione (solo per built-in)
static int __init hello_init(void)
{
    pr_info("Modulo caricato!\n");
    return 0; // 0 = successo, valore negativo = errore
}

// Funzione di cleanup - chiamata alla rimozione
// __exit: non viene nemmeno compilata se il modulo è built-in
static void __exit hello_exit(void)
{
    pr_info("Modulo rimosso!\n");
}

// Macro per registrare le funzioni
module_init(hello_init);
module_exit(hello_exit);

// Metadata obbligatori
MODULE_LICENSE("GPL");      // IMPORTANTE: senza GPL, molte funzioni non sono a
                             ccessibili
MODULE_DESCRIPTION("Modulo di esempio");
MODULE_AUTHOR("Nome Cognome");
MODULE_VERSION("1.0");
```

3.2 Integrazione nel Build System (Kbuild)

Il kernel usa un sistema di build chiamato **Kbuild**. Per aggiungere un modulo ci sono due approcci:

Opzione A: Creare una nuova sottodirectory

Struttura:

```
drivers/
├── mio_modulo/
│   ├── Kconfig      # Definisce l'opzione di configurazione
│   ├── Makefile     # Regole di compilazione
│   └── mio_modulo.c  # Codice sorgente
```

1. Kconfig (`drivers/mio_modulo/Kconfig`):

```
menu "Menu per il mio modulo"

config MIO_MODULO
    tristate "Il mio modulo custom"
```

```
help
    Descrizione del modulo.
    Appare in make menuconfig.

endmenu
```

2. Makefile (`drivers/mio_modulo/Makefile`):

```
obj-$(CONFIG_MIO_MODULO) += mio_modulo.o
```

3. Collegamento al padre (`drivers/Kconfig`):

```
source "drivers/mio_modulo/Kconfig"
```

4. Collegamento al padre (`drivers/Makefile`):

```
obj-$(CONFIG_MIO_MODULO) += mio_modulo/
```

Opzione B: Aggiungere a una directory esistente

Più semplice: basta modificare Kconfig e Makefile esistenti nella directory.

Compilazione come modulo esterno (Out-of-tree)

Per sviluppo rapido, senza modificare l'albero dei sorgenti:

```
# Makefile esterno
obj-m := mio_modulo.o
KDIR := /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

```
make          # Compila
sudo insmod mio_modulo.ko # Carica
sudo rmmod mio_modulo    # Rimuove
```

3.3 Caricamento Automatico dei Moduli

Il kernel può caricare automaticamente i moduli quando ne ha bisogno. Esempio classico: **I/O Scheduler**.

```
# Verifica scheduler disponibili
cat /sys/block/sda/queue/scheduler
# Output: [mq-deadline] kyber bfq

# Se bfq è un modulo e non è caricato, non appare. Ma:
echo bfq | sudo tee /sys/block/sda/queue/scheduler
# Il kernel carica automaticamente il modulo bfq!

lsmod | grep bfq
# bfq    106496 1
```

3.4 Il Tempo nel Kernel: HZ e Jiffies

Il kernel non opera in modo continuo, ma a **impulsi** chiamati **tick**.

La costante HZ

HZ definisce quanti tick avvengono al secondo. Valori comuni:

HZ	Periodo Tick	Uso Tipico	Pro	Contro
100	10 ms	Server	Basso overhead	Alta latenza
250	4 ms	Default molte distro	Bilanciato	-
1000	1 ms	Desktop/Gaming	Bassa latenza	Alto overhead

La variabile jiffies

jiffies è un contatore globale incrementato ad ogni tick. È l'“orologio interno” del kernel.

```
unsigned long now = jiffies;
// Dopo 1 secondo:
// now + HZ == jiffies (circa)
```

Conversioni utili:

```
msecs_to_jiffies(100) // 100ms in jiffies
jiffies_to_msecs(j)   // jiffies in milliseconds
```

Implicazioni pratiche

- Lo **scheduler** controlla ad ogni tick se è ora di cambiare processo
- I **timer** del kernel hanno granularità pari a $1/\text{HZ}$ secondi
- Operazioni time-critical potrebbero richiedere timer ad alta risoluzione (hrtimers)

Strumenti di misura:

- **cyclicttest**: Misura latenza risposta a interrupt

- `top` : Verifica overhead (tempo in sys/idle)

PARTE III: Interazione e Debugging

Modulo 04: Kernel OOPS, Debugging e Sysfs

4.1 Il Kernel Log

Il kernel scrive i suoi messaggi in un **ring buffer** (buffer circolare). Questi messaggi sono fondamentali per il debugging.

Accesso ai log:

```
dmesg          # Mostra tutto il buffer
dmesg -H        # Formato human-readable
dmesg -w        # Follow (come tail -f)
dmesg -l err,warn # Solo errori e warning
journalctl -k   # Alternativa con systemd
```

Livelli di log (da più a meno grave):

Livello	Macro	Descrizione
0	<code>pr_emerg</code>	Sistema inutilizzabile
1	<code>pr_alert</code>	Azione immediata richiesta
2	<code>pr_crit</code>	Condizione critica
3	<code>pr_err</code>	Errore
4	<code>pr_warn</code>	Warning
5	<code>pr_notice</code>	Normale ma significativo
6	<code>pr_info</code>	Informativo
7	<code>pr_debug</code>	Debug (solo se DEBUG abilitato)

Redirect su porta seriale (per VM):

```
# In /etc/default/grub:
GRUB_CMDLINE_LINUX="console=ttyS0 ignore_loglevel"
sudo update-grub
```

Netconsole (invio log via rete):

```
sudo modprobe netconsole netconsole="@/eth0,@192.168.1.100/"
```

4.2 Kernel OOPS e Panic

Cos'è un OOPS?

Un **OOPS** è un errore rilevato dal kernel (es. dereferenza NULL pointer, divisione per zero). Non è necessariamente fatale.

```
[ 116.960673] BUG: kernel NULL pointer dereference, address: 000000000000001a8
[ 116.962021] #PF: supervisor read access in kernel mode
...
[ 116.970105] RIP: 0010:bfq_bfqq_save_state+0x7d/0x890 [bfq]
...
[ 116.973829] Call Trace:
[ 116.985016] bfq_merge_bfqqs+0x192/0x1180 [bfq]
...
```

OOPS vs Panic

Aspetto	OOPS	Panic
Gravità	Errore recuperabile	Errore fatale
Effetto	Uccide il processo colpevole	Blocca l'intero sistema
Sistema	Continua (ma "tainted")	Richiede reboot
Causa tipica	Bug in modulo	Bug in codice critico (scheduler, interrupt)

Analisi di un OOPS

Informazioni chiave:

1. **RIP** (Instruction Pointer): `funzione+offset/lunghezza`
2. **Call Trace**: Stack delle chiamate
3. **Registri**: Stato della CPU al momento del crash

Debugging con GDB:

```
# Compilare con DEBUG_INFO=y
gdb vmlinux # o il file .ko del modulo
(gdb) list *(bfq_bfqq_save_state+0x7d)
```

4.3 Debugging Proattivo: BUG_ON e WARN_ON

Offensive Programming: Non aspettare che i bug si nascondano. Falli esplodere subito!

```
// BUG_ON: Se la condizione è vera → OOPS (kill processo)
BUG_ON(ptr == NULL);

// WARN_ON: Se la condizione è vera → Stack trace (senza kill)
WARN_ON(count < 0);

// WARN_ONCE: Come WARN_ON ma stampa solo la prima volta
WARN_ONCE(count < 0, "Contatore negativo!");
```

Perché usarli?

- Un crash immediato vicino al bug è più facile da debuggare
- Una corruzione silenziosa può manifestarsi molto dopo, in punti completamente diversi

4.4 Sysfs: Interfaccia tra Kernel e Userspace

Sysfs (`/sys`) è un filesystem virtuale che espone la struttura interna del kernel allo userspace.

Struttura concettuale

```
/sys
├── block/      # Dispositivi a blocchi
├── bus/        # Bus (PCI, USB, etc.)
├── class/      # Classi di dispositivi
├── devices/    # Albero dei dispositivi
├── firmware/   # Interfacce firmware
├── fs/         # Filesystem
├── kernel/     # Parametri kernel
└── module/     # Moduli caricati
```

Kobject e Attributi

- **Kobject:** Struttura C nel kernel → directory in `/sys`
- **Attributo:** Campo del kobject → file in `/sys`

Creare un file sysfs:

```
static ssize_t my_show(struct kobject *kobj, struct kobj_attribute *attr, char *buf)
{
    return sprintf(buf, "Hello World\n");
}

static ssize_t my_store(struct kobject *kobj, struct kobj_attribute *attr,
                        const char *buf, size_t count)
{
    pr_info("Ricevuto: %s\n", buf);
    return count;
}

static struct kobj_attribute my_attr = __ATTR(my_file, 0644, my_show, my_store);
```

Race Conditions

Problema: Se due processi accedono contemporaneamente a una variabile via sysfs, senza sincronizzazione si hanno race conditions.

```
// SBAGLIATO - race condition
static int counter = 0;
static ssize_t show(...)
{
    return sprintf(buf, "%d\n", counter++);
}

// CORRETTO - uso di spinlock
static DEFINE_SPINLOCK(lock);
static ssize_t show(...)
{
    int val;
    spin_lock(&lock);
    val = counter++;
    spin_unlock(&lock);
    return sprintf(buf, "%d\n", val);
}
```

Modulo 05: System Call e Tracing

5.1 System Call: L'Interfaccia Kernel/Userspace

Le **system call** sono il meccanismo attraverso cui i programmi userspace richiedono servizi al kernel.

flowchart TD

A["Processo Utente
es. read()"] →|syscall| B["Kernel
es. sys_read()"]

Aggiungere una nuova System Call

1. Definire la funzione (`kernel/mysyscall.c`):

```
#include <linux/kernel.h>
#include <linux/syscalls.h>
SYSCALL_DEFINE0(mysyscall){
    pr_info("mysyscall invocata!\n");
    return 0;
}
```

2. Aggiungere al build system (`kernel/Makefile`):

```
obj-y += mysyscall.o
```

3. Dichiarare il prototipo (`include/linux/syscalls.h`):

```
asmlinkage long sys_mysyscall(void);
```

4. Registrare nella tabella ([arch/x86/entry/syscalls/syscall_64.tbl](#)):

```
470 common mysyscall sys_mysyscall
```

5. Test da userspace:

```
#include <sys/syscall.h>#include <unistd.h>#include <stdio.h>int main() { long ret = s
yscall(470); printf("Syscall returned: %ld\n", ret); return 0;}
```

5.2 Tracing: Vedere Cosa Succede nel Sistema

strace: Tracing Userspace

Mostra le system call invocate da un processo:

```
strace ls
# Output esempio:
# open(".", O_RDONLY|O_NONBLOCK|O_DIRECTORY) = 3
# getdents64(3, ...) = 344
# write(1, "file1\nfile2\n", 12) = 12

# Statistiche aggregate
strace -c ls

# Filtrare solo syscall "open"
strace -e open ls

# Attach a processo esistente
strace -p &lt;PID>
```

ftrace: Tracing Kernel

Il tracer interno del kernel, accessibile via filesystem:

```
# Interfaccia
/sys/kernel/tracing/      # o /sys/kernel/debug/tracing/

# Abilitare il tracing di funzioni
echo function &gt; /sys/kernel/tracing/current_tracer
echo 1 &gt; /sys/kernel/tracing/tracing_on
cat /sys/kernel/tracing/trace
```

```
# Tracciare una specifica funzione
echo schedule &gt; /sys/kernel/tracing/set_ftrace_filter
```

trace-cmd: Frontend per ftrace

```
# Registrare eventi
trace-cmd record -e sched_switch -e sched_wakeup sleep 1

# Visualizzare
trace-cmd report
```

5.3 I/O Tracing con blktrace

Per analizzare le performance del disco:

Preparazione:

```
# Svuotare le cache per risultati ripetibili
sudo sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Analisi aggregata:

```
iostat -x 1      # Statistiche ogni secondo
```

Analisi dettagliata:

```
sudo blktrace -d /dev/sda -o - | blkparse -i -
```

Interpretazione eventi blktrace:

Evento	Significato
Q	Richiesta in coda
G	Richiesta ottenuta
I	Inserita nello scheduler
D	Inviata al driver
C	Completata

PARTE IV: Gestione del Lavoro

Modulo 06: Git e Code Management

6.1 Comandi Git Essenziali

Navigazione e Stato

```
git status          # Stato attuale
git log --oneline -n 10  # Ultimi 10 commit
git show <commit>      # Dettagli di un commit
git diff            # Modifiche non staged
git diff --cached      # Modifiche staged
```

Gestione Branch

```
git branch          # Lista branch
git checkout <nome>  # Cambia branch
git checkout -b <nuovo> # Crea e cambia
git branch -D <nome> # Elimina branch
```

Commit

```
git add <file>      # Stage file
git add -p          # Stage interattivo (per hunk)
git commit          # Commit staged
git commit -a       # Add + commit tracked files
git commit --amend   # Modifica ultimo commit
```

Reset e Pulizia

```
git reset --hard HEAD    # Scarta tutte le modifiche
git reset --hard <commit> # Torna a un commit (DISTRUTTIVO)
git clean -fd            # Rimuove file non tracciati
git reflog               # Storia dei movimenti (per recuperare)
```

Stash: Modifiche Temporanee

```
git stash          # Salva modifiche
git stash list     # Lista stash
git stash pop      # Recupera e rimuove
git stash apply    # Recupera senza rimuovere
```

Rebase Interattivo

```
git rebase -i HEAD~5    # Modifica ultimi 5 commit
```

Opzioni:

- **pick** : Mantieni commit
- **reword** : Cambia messaggio
- **edit** : Modifica contenuto
- **squash** : Fondi con precedente
- **drop** : Elimina

Cherry-pick

```
git cherry-pick <commit>    # Applica un commit da altro branch
```

6.3 Workflow del Kernel Linux

Creare una Patch

```
git format-patch HEAD~1      # Ultima commit
git format-patch --cover-letter HEAD~3    # Ultimi 3 + cover letter
```

Verificare la Patch

```
./scripts/checkpatch.pl 0001-my-patch.patch
```

Inviare la Patch

```
# Configurazione SMTP in ~/.gitconfig
[sendemail]
    smtpserver = smtp.gmail.com
    smtpserverport = 465
    smtpencryption = ssl
    smtpuser = tuaemail@gmail.com

# Trovare i destinatari
./scripts/get_maintainer.pl 0001-my-patch.patch

# Inviare
git send-email 0001-my-patch.patch --to=maintainer@example.com --cc=list@vger.kernel.org
```

6.4 Best Practices per i Commit

Struttura del Messaggio

<sottosistema>: <descrizione imperativa breve>

Corpo del messaggio:

- Spiega il PERCHÉ, non il COSA (il cosa si vede dal codice)
- Contesto del problema
- Motivazione della soluzione
- Eventuali effetti collaterali

Signed-off-by: Nome Cognome <email@example.com>

Esempi

✓ **Corretto:**

✗ **Sbagliato:**

block, bfq: fix memory leak in bfqq merge

Fixed bug

When merging two bfq_queues, the reference count of the victim queue was not decremented, causing a memory leak.

This patch adds the missing bfq_put_queue() call.

Signed-off-by: Paolo Valente <paolo.valente@linaro.org>

6.5 Compilazione Incrementale

Problema: Cambiando branch, git aggiorna i timestamp di tutti i file modificati. Make potrebbe ricompilare tutto.

Soluzione (se si è sicuri che i .o sono corretti):

```
make -t O=<build_dir> # Touch fittizio dei file
```

⚠ Non funziona se CONFIG_STACK_VALIDATION è abilitato (objtool ricompila comunque).

Appendice: Comandi Rapidi

Compilazione Kernel

```
make defconfig && ./scripts/kconfig/streamline_config.pl > .config  
make -j $(($(nproc) + 1))sudo make modules_install && sudo make install  
sudo update-grub && sudo reboot
```

Gestione Moduli

```
lsmod | grep <nome>      # Cerca modulo  
sudo modprobe <nome>     # Carica  
sudo modprobe -r <nome>  # Rimuove  
modinfo <nome>          # Info
```

Debugging

```
dmesg -wH                # Follow log  
cat /sys/kernel/debug/tracing/trace # ftrace  
strace -p <PID>          # Trace syscall
```

Git Kernel

```
git format-patch HEAD~1    # Crea patch  
./scripts/checkpatch.pl *.patch # Verifica  
git send-email *.patch     # Invia
```

Buono Studio! 🎓

Ricorda: nel kernel hacking la pratica è tutto. Compila, rompi, debugga, impara.